# LC → 86 → Partition list

→ linked list ka head diya hai
→ aur value (x).
→ partition in such that way, all nodes less than (x).

x = 3 [1 → 4 → 3 → 2 → 5 → 2]

→ [1 → 2 → 2 → 3 → 4 → 5]

x = 2 , head = [2, 1]

[1 → 2] (2 partitions)

**CORE Concept:**

(2) list banano
(1) val < 3
val >= 3
end mai jod do

IMP

[1 → 4 → 3 → 2 → 5 → 2]

if (node → val < 3) OK
else ↓ swap, with that node to end.

1 → 4 → 3 → 2 → 5 → 2.

1 → 3 → 4 → 2 → 5 → 2

1 → 3 → 2 → 4 → 5 → 2

1 → 4 → 3 → 2 → 5 → 2

iterate throughit

list (1) = 1 → 2 → 2

list (2) = 4 → 3 → 5

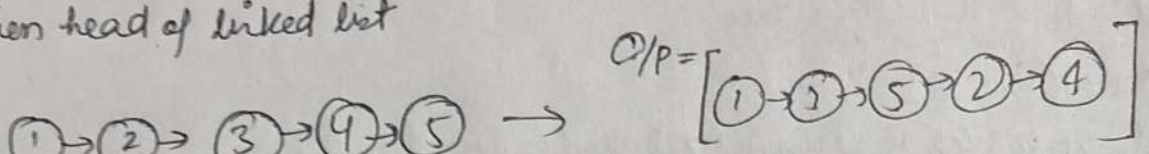
merge then

(i) list1 ko tail tk iteration
(ii) tail → next = head 2
(iii) Return list1 head ;

TC → O(n)
SC → O(n) + O(m) = O(n + m)
= O(n)

# LC→328 Odd Even Linked List :

given head of linked list

→ 


1 2 3 4 5 → reorder such that odd indices comes first

Approach → take all odd indices and create list 1
take all even indices and create list 2
→ merge oddtail → next = evenhead;
return even head,

here TC → O(n) since Sigle traversal / SC → O(n) → new list formed

---

Code;

```
LN* oddhead = NULL, oddtail, evenhead, eventail = NULL;
count = 1;
LN* temp = head;
while(temp)
{
  if(count%2 != 0)
  {
    // odd mai add;
  }
  else {
    // even mai add;
  }
  count++;
  temp = temp → next;
}
if (Oddhead == NULL) return evenhead;
if (evenhead == NULL) return oddhead;
oddtail → evenhead;
return evenhead;
```

But interview needs it in SC → O(1)
→ using dummy node

S(→O(1)  LN° LD(0), MD(0);

```
ListNode* less = &lessdummy;
ListNode* more = &moredummy;
int cont=1;
while(head)
{
    if(cont%2!=0)
    {
        less->next=head;
        less = less->next;
    }
    else{
        more->next=head;
        more = more->next;
    }
    Cont++;
    head=head->next;
}
more->next=NULL;
less->next = moredummy.next;   } cause these are pointer)
return lessdummy.next;
}
```

LC → 25

# Reverse Nodes K-group

→ Given 'k' integer, Reverse Nodes in 'k' groups.

→ Idea → Reverse krna hai;
bus groups mai;

① Vector mai push krke, fir reverse krke, pi LL Banado.

```
# if(head == NULL || k == 1)        // same rehna hai
    return head;

vector<int> v;
ListNode* temp = head;
while(temp)
{
    ans.push_back(temp->val);
    temp = temp->next;
}

//reversal of k groups
for(int i=0; i+k<=v.size(); i+k)
{
    reverse(v.begin()+i(), v.end()+i+k);
}

temp = head;
int i=0;
while(temp){
    temp->val = v[i++];
    temp = temp->next;
}
return head;
}
```

Isme, Vector increases the time complexity() but is ok to start with brute.

SC → O(n)

(2) Use a stack, push (k) elements (values)
→ pop them & make new linked list.

```
# if(head == NULL || k==1) return head;
  Stack<int> S;
LN* temp = head;
"   newhead = NULL;
"    tail = NULL;
while(temp)
{
  int cnt=0;
  LN* curr = temp;
  while(curr && cnt<k)
  {
    S.push(curr->val);
    curr = curr->next;
    cnt++;
  }
  if(cnt==k) {                    // k filled
    while(!S.empty())
    {
      LN* newnode = new LN(S.top());
      S.pop();
      if(!head) { newhead = tail = newnode;
      else { tail->next = newnode, tail = newnode;
    }
  }
  else {                          // less element
    while(temp != curr)
    {
      =
      =
    }
    break;
    temp = curr;
  }
return newhead;
```

① → ② → ③ → ④ → ⑤ → ⑥ ;

# Pointer Approach

→ When we try to solve problem by changing the (next pointer) of each node,
→ without altering the value of node.

(node → next = kisiaurnode)

→ Why ? → to avoid stack / vectors
→ Extra memory
→ New list banna.

} with it ? → In place solution
SC → O(1)

Linked list is a powerful game with Pointer manipulation

## Fundamentals :

① Create diagram
② Handle curr, next, prev pointer
③ no use of value only Node
④ Never forget Head (Dummy, newhead)

## Types :

① Single traversal
(for length, search)
while(temp) temp = temp → next;

② Rearranging links (odd-even)
odd → next = even → next;
odd = odd → next;

③ Reversal,
next = curr → next;
curr → next = prev;
prev = curr;
curr = next;

---

**Defination** : In modifying the link in place without using extra memory where using vector uses extra space, to reduce the (SC) we have in place version possible.